Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(Университет ИТМО)

Факультет прикладной информатики
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии

О Т Ч Е Т
по лабораторной работе № 4

Дисциплина: Метод множителей Лагранжа
Тема: «Динамическое программирование»

Обучающийся: Данилова Айаана Васильевна, группа K3341, поток 1.4
Преподаватель: Микрюкова А.С.

Санкт-Петербург 2026

ЦЕЛЬ РАБОТЫ

Решение задачи оптимизации методом динамического программирования.

Вариант работы: 10

Задача 10.

Предприятие распределяет 10 одинаковых пакетов инвестиций между тремя мастерскими. Прирост выпуска каждой мастерской указан в таблице. Финансирование могут получить не более двух мастерских. Требуется распределить все 10 единиц ресурса так, чтобы получить максимальный суммарный прирост выпуска.

u	Мастерская А	Мастерская Б	Мастерская В
0	0	0	0
1	5	3	4
2	9	8	10
3	13	14	13
4	16	18	15
5	18	21	17
6	20	23	19
7	21	24	20
8	22	25	21
9	23	26	22
10	24	27	23

ХОД РАБОТЫ

В процессе выполнения работы было проведено поэтапное ручное решение задачи распределения инвестиций методом динамического программирования с построением рекуррентных соотношений Беллмана и таблиц состояний. Затем на языке Python был реализован программный алгоритм с использованием класса DPSolver, позволивший оцифровать процесс решения, учесть установленные ограничения на количество финансируемых объектов и автоматизировать поиск оптимального распределения ресурсов.

№10

10 пакетов инвестиций. 3 мастерских.

1) Мат постановка

u_i - кол-во инвестиций, выделенных на i-ую мастерскую

$r_i(u_i)$ - прирост при выделении u_i пакетов i

$i \in \{1..3\}$.

Ограничения:

$\|u\|_0 \leq 2$ т.е кол-во ненулевых u_i равно 2.

На сами пакеты по условию нет ограничений.

Тогда итоговая функция имеет вид:

$$F = r_1(u_1) + r_2(u_2) + r_3(u_3) \to max$$

$$u_1 + u_2 + u_3 = 10$$

$$u = (u_1, u_2, u_3)$$

$$\|u\|_0 \leq 2.$$

2) Рекуррентное соотнош-ие Беллмана и этапы.

Вводится счетчик K - число объектов отличных от

$$V_i(s, K) = \max_{0 \leq u \leq s} \left[r_i(u) + V(s-u, K-1 [u>0]) \right]$$

8 0,1,2,3,4,5,6,7,8
9 0,1,3,4,5,6,7,8,9
10 0,1,2,3,4,5,6,7,8,9,10

3) знаки 0, 1, 2

2

S	$V_2(S,0)$	$V_2(S,1)$	$V_2(S,2)$
0	0	0	0
1	4	4	
2	10	10	
3	13	13	
4	15	15	
5	17	17	—
6	19	19	
7	20	20	
8	21	21	
9	22	22	
10	23	23	

1

S	u	$M_1(u)$	$V_1(S,0)$	$V_1(S,1)$	$V_1(S,2)$
0	0	0,4	0	0	невозможно
1	0,1	0,3	4	4	
2	0,1,2	0,3,8	10	10	
3	0,1,2,3	0,3,8,14	14	14	
4	0,1,2,3,4	0,3,8,14,18	18	18	—
5	0,1,2,3,4,5	0,3,8,14,18,21	24	(21)	
6	0,1,2,3,4,5,6	0,3,8,14,18,21,23	28	(23)	
7	0,1,2,3,4,5,6,7	0,3,8,14,18,21,23,24	31	24	
8	0,1,2,3,4,5,6,7,8	0,3,8,14,18,21,23,24,25	34	25	
9	0,1,2,3,4,5,6,7,8,9	0,3,8,14,18,21,23,24,25,26	36	26	
10	0,1,2,3,4,5,6,7,8,9,10	0,3,8,14,18,21,23,24,25,26,27	38	27	

Примеры вычислений

$V_1(0,0) = M_1(0) + V_2(0,0)$

$V_1(1,0) = \max[M_1(0) + V_2(1,0),\ M_1(1) + V_2(0,1)] = \max[4, 3] = 4$

$V_1(2,0) = \max[M_1(0) + V_2(2,0),\ M_1(1) + V_2(1,1),\ M_1(2) + V_2(0,1)] = \max[10,$

$V_1(3,0) = \max[13, 16, 12, 14] = 14$ $V_1(4,0) = \max[15, 16, 18, 18, 18] = 18$

$V_1(5,0) = \max[17, 18, 21, 24, 22, 21] = 24$ $V_1(6,0) = \max[19, 20, 23, 24, 22, 25, 23] = 28$ и

$V_1(0,1) = r_1(0) + V_2(0,1) = 0$

$V_1(1,1) = \max[0 + V_2(1,1),\ 3 + V_2(0,2)] = \max[4, 3] = 4$

$V_1(2,1) = \max[0 + V_2(2,1),\ 8 + V_2(0,2)] = \max[10, 8] = 10$

$V_2(3,1) = \max[13, 14] = 14 \quad V_2(4,1) = \max[$

3)

S^1	u	$r_0(u)$	$V_0(10,0)$
9	0	0	38
	1	5	31
	2	9	34
	3	13	36
10	4	16	39
	5	18	max 39 = 39
	6	20	38
	7	21	35
	8	22	32
	9	23	27
	10	24	24

$V_0(10,0) = \max[r_0(0) + V_1(10,0),\ r_0(1) + V_1(9,1),\ r_0(2) + V_1(8,1),$
$r_0(3) + V_1(7,1),\ r_0(4) + V_1(6,1),\ r_0(5) + V_1(5,1),\ r_0(6) + V_1(4,1),$
$r_0(7) + V_1(3,1),\ r_0(8) + V_1(2,1),\ r_0(9) + V_1(1,1),\ r_0(10) + V_1(0,1)]$
$= \max[38, 31, 34, 36, 39, 39, 38, 35, 32, 27, 24] = 39$

4) Восстановление оптимальной стратегии

Результат 39 получился в 2 ситуациях:

↳ выделено 4 пакета ~~[illegible]~~ М-А

↳ выделено 5 пакетов М-А

Т.е. смотрим $V_1(6,1)$ и $V_1(5,1)$ видим их соответствие

$\parallel \qquad \parallel$

$r_1(6) + V_2(0,2) \qquad r_1(5) + V_2(0,2)$

Т.е. реш-ия $(4, 6, 0)$ и $(5, 5, 0)$

Отв: макс значение 39 при u_1

Проанализировав все постановки заданий из файла, посвященной ДП, было решено что решение задач должно охватывать/учитывать следующие ограничения:

– относящиеся к n_i, как к конкретному, так и некоторой группе.

Пример: $n_i \geq 1$; $n_{k_1} \geq 2$ $n_{k_2} \geq 3$; $n_i \,\%\, 2 == 0$ и т.д. В целом можно собрать в 1 функцию, возвр. булево значение все подобные условия для каждого n_i. Не хранимая ~~filters~~: list[lambda n_i: ...]

– также ограничение кол-ва объектов в целом, т.е $\|n\|_0 \leq t$, где $0 < t \leq S$, $S \in \mathbb{N}$.

Для того чтобы отсеивать неподходящие решения будет использоваться флаг того что такое ограничение вообще существует и счётчик ненулевых n_i.

– глобальной характер задачи: max и min.

В зависимости от типа определяются дефолтные значения и операция сравнения.

Псевдокод:

DPInput (n, m, data, filters, limit_t)

DPSolver:

def solve (task: DPInput):

initialization all properties

if limit_t:

solve_with_limit();

else:

solve_without_limit();

def solve_with_limit():

V = [[[[default] * (t+1)] * (n+1)] * (m+1)]

↑ 3мерный контейнер решений.

choises ← для восстановления

for i ∈ [m-1..0]:

for s ∈ [0..m]:

best_u, best_value

for u ∈ [0..s]:

if !all(filters(u)):

continue

k += (u != 0)

if k > t: continue

обновление best_u, best_val

если

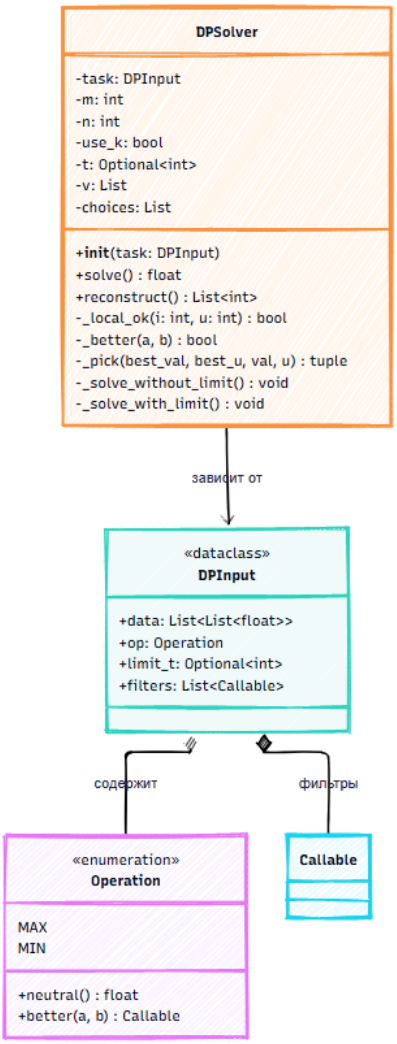
или

def solve_without_limit:

по не самое что с лимитом но только без лишнего уровня цикла и размерности

```
def reconstruct
    res
    for i ∈ [0..m):
        if limit_t != None:
            u = self.choises[i][s][k]
            k += (u != 0)
        else:
            u.choises[i][s]
            res.append(w); s -= u
```

Диаграмма классов программы:



Листинг:

```
from enum import Enum
import operator

class Operation(Enum):
    MAX = "max"
    MIN = "min"

    @property
    def neutral(self) -> float:
        return float('-inf') if self == Operation.MAX else float('inf')

    @property
    def better(self):
        return operator.gt if self == Operation.MAX else operator.lt
```

```python
from dataclasses import dataclass, field
from typing import List, Callable, Optional

@dataclass
class DPInput:
    data: List[List[float]]
    op: Operation = Operation.MAX
    limit_t: Optional[int] = None
    filters: List[Callable[[int], bool]] = field(default_factory=list)

from typing import Optional


class DPSolver:
    def __init__(self, task: 'DPInput'):
        self.task = task
        self.m = len(task.data)
        self.n = len(task.data[0]) - 1

        self.use_k: bool = task.limit_t is not None
        self.t: Optional[int] = task.limit_t

        self.v = None
        self.choices = None


    def _local_ok(self, i: int, u: int) -> bool:
        """локальный фильтр для объектов"""
        if i < len(self.task.filters):
            return self.task.filters[i](u)
        return True

    def _better(self, a, b) -> bool:
        """a лучше b согласно OP (max/min)"""
        return self.task.op.better(a, b)

    def _pick(self, best_val, best_u, val, u):
        """обновление лучшего кандидата"""
        if best_u is None or self._better(val, best_val):
            return u, val
        return best_u, best_val


    def solve(self):
        m, n = self.m, self.n
```

```
        OP = self.task.op

        if self.use_k:
            self._solve_with_limit()
        else:
            self._solve_without_limit()

        return self.v[0][n][0] if self.use_k else self.v[0][n]


    def _solve_without_limit(self):
        """2D-ДП: V[i][s]"""
        m, n, OP = self.m, self.n, self.task.op

        self.v = [[OP.neutral] * (n + 1) for _ in range(m + 1)]
        self.choices = [[None] * (n + 1) for _ in range(m)]

        for s in range(n + 1):
            self.v[m][s] = 0

        for i in range(m - 1, -1, -1):
            for s in range(n + 1):
                best_u, best_val = None, OP.neutral
                for u in range(s + 1):
                    if not self._local_ok(i, u):
                        continue
                    current = self.task.data[i][u] + self.v[i + 1][s -
u]
                    best_u, best_val = self._pick(best_val, best_u,
current, u)
                self.v[i][s] = best_val
                self.choices[i][s] = best_u


    def _solve_with_limit(self):
        """3D-ДП: V[i][s][k], где k число уже задействованных
объектов"""
        m, n, t, OP = self.m, self.n, self.t, self.task.op

        self.v = [[[OP.neutral] * (t + 1) for _ in range(n + 1)] for _
in range(m + 1)]
        self.choices = [[[None] * (t + 1) for _ in range(n + 1)] for _
in range(m)]

        for s in range(n + 1):
            for k in range(t + 1):
```

```
                self.v[m][s][k] = 0

        for i in range(m - 1, -1, -1):
            for s in range(n + 1):
                for k in range(t + 1):
                    best_u, best_val = None, OP.neutral
                    for u in range(s + 1):
                        if not self._local_ok(i, u):
                            continue
                        nz = 1 if u > 0 else 0
                        k_new = k + nz
                        if k_new > t:
                            continue
                        current = self.task.data[i][u] + self.v[i + 1][s
- u][k_new]
                        best_u, best_val = self._pick(best_val, best_u,
current, u)
                    self.v[i][s][k] = best_val
                    self.choices[i][s][k] = best_u


    def reconstruct(self):
        if self.choices is None:
            raise RuntimeError("Сначала вызовите solve()")

        s = self.n
        k = 0
        solution = []
        for i in range(self.m):
            if self.use_k:
                u = self.choices[i][s][k]
                k += 1 if u > 0 else 0
            else:
                u = self.choices[i][s]
            solution.append(u)
            s -= u
        return solution

data = [
    [0, 5, 9, 13, 16, 18, 20, 21, 22, 23, 24], # A
    [0, 3, 8, 14, 18, 21, 23, 24, 25, 26, 27], # Б
    [0, 4, 10, 13, 15, 17, 19, 20, 21, 22, 23] # В
]

task = DPInput(
    data=data,
```

```
    op=Operation.MAX,
    limit_t=2,
    filters=[lambda u: True, lambda u: True, lambda u: True]
)
solver = DPSolver(task)
max_value = solver.solve()
solution = solver.reconstruct()
print(f"Максимум: {max_value}, Решение: {solution}")
```

ВЫВОДЫ

В ходе выполнения лабораторной работы была успешно продемонстрирована высокая эффективность метода динамического программирования для решения задач оптимального распределения ресурсов. Разработанный программный класс DPSolver объединил подходы двумерного (V[i][s]) и трехмерного (V[i][s][k]) динамического программирования, позволяя гибко учитывать ограничение limit_t на количество финансируемых объектов и пользовательские локальные фильтры. Использование табличного заполнения (мемоизации) в сочетании с предварительным отсечением недопустимых состояний через фильтрацию ограничений эффективно сокращает перебираемое пространство решений. Несмотря на наличие многоуровневых вложенных циклов, такой подход исключает повторные вычисления, что обеспечивает высокую вычислительную скорость, хорошую масштабируемость и практическую применимость алгоритма.

ВИДЕО-РЕФЛЕКСИЯ

Ссылка на видео-рефлексию: 4.MOV